

CÁC PHÉP TOÁN VÀ BIẾN TRONG PYTHON

Several thin, parallel white lines of varying lengths and slopes are positioned on the right side of the slide, extending from the top right towards the bottom left.

1. CÁC PHÉP TOÁN TRONG PYTHON

Phép toán (hay Toán tử) là các cấu trúc có thể thao tác giá trị của toán hạng (hay số hạng).

Xét biểu thức $4 + 5 = 9$. Ở đây, 4 và 5 được gọi là toán hạng và + được gọi là phép toán.

Các loại phép toán trong Python

Ngôn ngữ Python hỗ trợ các loại phép toán sau.

- Phép toán số học
- Phép toán so sánh (quan hệ)
- Phép toán chuyển nhượng
- Phép toán logic
- Phép toán bitwise (phép toán nhị phân)
- Phép toán dịch bit
- Phép toán nhận dạng

1.1. PHÉP TOÁN SỐ HỌC TRONG PYTHON

Giả sử biến a giữ 10 và biến b giữ 20, thì:

Phép toán	Mô tả	Ví dụ
+	Cộng giá trị ở hai bên của toán tử.	$a + b = 30$
-	Trừ toán hạng bên trái cho toán hạng bên phải.	$a - b = -10$
*	Nhân hai toán tử cho nhau.	$a * b = 200$
/	Chia toán hạng bên trái cho toán hạng bên phải	$b / a = 2$
%	Chia lấy phần dư toán hạng bên trái cho toán hạng bên phải	$b \% a = 0$
**	Tính lũy thừa trên các toán tử	$a ** b = 10 \text{ lũy thừa } 20$
//	Chia lấy cận dưới (hay chia lấy phần nguyên) - Chia toán hạng trong đó kết quả là thương số trong đó các chữ số sau dấu thập phân được loại bỏ. Nhưng nếu một trong các toán hạng là âm, kết quả sẽ được thả nổi, tức là được làm tròn từ 0 (về phía vô cực âm)	$9//2=4$ và $9.0//2.0=4.0$, $-11//3=-4$, $-11.0//3=-4.0$

1.2. PHÉP TOÁN SO SÁNH TRONG PYTHON

Giả sử biến a giữ 10 và biến b giữ 20, thì:

Phép toán	Mô tả	Ví dụ
==	Nếu các giá trị của hai toán hạng bằng nhau, thì <u>điều kiện</u> trở thành True.	(a == b) False.
!=	Nếu giá trị của hai toán hạng không bằng nhau, thì điều kiện trở thành True.	(a != b) True.
<>	Nếu giá trị của hai toán hạng không bằng nhau, thì điều kiện trở thành True.	(a <> b) True. Điều này tương tự như phép toán !=
>	Nếu giá trị của toán hạng bên trái lớn hơn giá trị của toán hạng bên phải, thì điều kiện sẽ trở thành True.	(a>b) False.
<	Nếu giá trị của toán hạng bên trái nhỏ hơn giá trị của toán hạng bên phải, thì điều kiện sẽ trở thành True.	(a<b) True.
>=	Nếu giá trị của toán hạng bên trái lớn hơn hoặc bằng giá trị của toán hạng bên phải, thì điều kiện sẽ trở thành True.	(a >= b) False.
<=	Nếu giá trị của toán hạng bên trái nhỏ hơn hoặc bằng giá trị của toán hạng bên phải, thì điều kiện sẽ trở thành True.	(a <= b) True.

1.3. PHÉP TOÁN GÁN TRONG PYTHON

Giả sử biến a giữ 10 và biến b giữ 20, thì:

Phép toán	Mô tả	Ví dụ
=	Gán các giá trị từ toán hạng bên phải cho toán hạng bên trái	$c = a + b$ gán giá trị của $a + b$ vào c
+=	Thêm toán hạng bên phải vào toán hạng bên trái và gán kết quả cho toán hạng bên trái	$c += a$ tương đương với $c = c + a$
-=	Trừ toán hạng bên phải từ toán hạng bên trái và gán kết quả cho toán hạng bên trái	$c -= a$ tương đương với $c = c - a$
*=	Nhân toán hạng bên phải với toán hạng bên trái và gán kết quả cho toán hạng bên trái	$c *= a$ tương đương với $c = c * a$
/=	Chia toán hạng bên trái với toán hạng bên phải và gán kết quả cho toán hạng bên trái	$c /= a$ tương đương với $c = c / a$
%=	Lấy mô-đun bằng hai toán hạng và gán kết quả cho toán hạng bên trái	$c \% = a$ tương đương với $c = c \% a$
**=	Thực hiện tính toán hàm mũ (lũy thừa) trên các toán tử và gán giá trị cho toán hạng bên trái	$c ** = a$ tương đương với $c = c ** a$
//=	Thực hiện phân chia tầng trên các toán tử và gán giá trị cho toán hạng bên trái	$c //= a$ tương đương với $c = c // a$

1.4. PHÉP NHỊ PHÂN VÀ PHÉP AND

Phép toán bitwise (phép toán nhị phân) của Python

Phép toán bitwise hoạt động trên bit và thực hiện thao tác bit by bit.

Giả sử nếu $a = 60$; và $b = 13$; bây giờ ở định dạng nhị phân, chúng sẽ như sau:

$a = 0011\ 1100$

$b = 0000\ 1101$

Ta có các phép toán bitwise sau được hỗ trợ bởi ngôn ngữ Python:

Phép toán & (AND) nhị phân:

Hoạt động theo nguyên tắc sau:

$0 \& 0 = 0$

$0 \& 1 = 0$

$1 \& 0 = 0$

$1 \& 1 = 1$

Theo nguyên tắc trên thì ta thấy rằng nếu 2 bit đều là 1 thì & với nhau sẽ được kết quả là 1, các trường hợp còn lại cho kết quả là 0.

1.5. PHÉP TOÁN | OR NHỊ PHÂN

Phép toán | (OR) nhị phân:

Hoạt động theo nguyên tắc sau:

$$0 | 0 = 0$$

$$0 | 1 = 1$$

$$1 | 0 = 1$$

$$1 | 1 = 1$$

Theo nguyên tắc trên thì nếu 2 bit đều là 0 thì | với nhau sẽ được kết quả là 0, các trường hợp còn lại sẽ cho kết quả là 1.

Phép toán ~ (NOT hay Đảo bit) nhị phân:

Hoạt động theo công thức sau:

$$\sim A = -(A + 1)$$

Trong đó A là một số hệ 10.

1.6. PHÉP TOÁN DỊCH TRÁI, DỊCH PHẢI NHỊ PHÂN

Phép toán << (Dịch trái) nhị phân

Hoạt động theo nguyên tắc sau:

Dịch trái biến a n bit tương ứng với: $a = a * 2^n$. Lưu ý a phải là một số nguyên.

Ví dụ:

$a = 60$

`print(a << 3)` #kết quả là 480

Phép toán >> (Dịch phải) nhị phân

Hoạt động theo nguyên tắc sau:

Dịch phải biến a n bit tương ứng với: $a = a // 2^n$. Lưu ý a phải là một số nguyên.

Ví dụ:

$a = 60$

`print(a >> 3)` #kết quả là 7

1.7. PHÉP TOÁN LOGIC TRONG PYTHON

Có các phép toán logic sau được hỗ trợ bởi ngôn ngữ Python. Giả sử biến a giữ 10 và biến b giữ 20 thì:

Toán tử	Sự miêu tả	Thí dụ
and	Nếu cả hai toán hạng đều đúng thì điều kiện trở thành True.	(a and b) True.
or	Nếu bất kỳ một trong hai toán hạng đều khác không thì điều kiện sẽ trở thành True.	(a or b) True.
not	Được sử dụng để đảo ngược trạng thái logic của toán hạng của nó.	not(a and b) False.

1.8. PHÉP TOÁN MEMBERSHIP TRONG PYTHON

Các phép toán membership của Python kiểm tra tư cách thành viên theo trình tự, chẳng hạn như chuỗi, danh sách hoặc bộ dữ liệu. Có hai phép toán membership như được giải thích dưới đây:

Phép toán	Mô tả	Ví dụ
in	Đánh giá là True nếu nó tìm thấy một biến trong chuỗi đã chỉ định, ngược lại là False.	<code>x in y</code> , ở đây kết quả là 1 nếu x là thành viên của chuỗi y.
not in	Đánh giá là True nếu không tìm thấy một biến trong chuỗi được chỉ định, ngược lại là False.	<code>x not in y</code> , ở đây không có kết quả là 1 nếu x không phải là thành viên của chuỗi y.

1.9. PHÉP TOÁN NHẬN DẠNG TRONG PYTHON

Phép toán nhận dạng so sánh các vị trí bộ nhớ của hai đối tượng. Có hai toán tử nhận dạng được giải thích như bên dưới:

Phép toán	Mô tả	Thí dụ
is	Đánh giá là True nếu các biến ở hai bên của toán tử trỏ đến cùng một đối tượng, ngược lại là False.	<pre>x = 2 print(x is 2) # sẽ in ra True y = 2.0 print(y is 2) # sẽ in ra False</pre>
is not	Đánh giá thành false nếu các biến ở hai bên của toán tử trỏ đến cùng một đối tượng và đúng khác.	<pre>x = 2 print(x is not 2) # sẽ in ra False y = 2.0 print(y is not 2) # sẽ in ra True</pre>

1.10. ĐỘ ƯU TIÊN PHÉP TOÁN TRONG PYTHON (1)

Độ ưu tiên	Toán tử & Mô tả
1	** Lũy thừa
2	~ + - Đảo bit, đảo dấu dương và đảo dấu âm
3	* / % // Nhân, chia, module và chia lấy nguyên
4	+ - Cộng và trừ
5	>> << Dịch bit phải và trái
6	& Bitwise 'AND'
7	^ Bitwise 'XOR' và 'OR' thông thường

1.10. ĐỘ ƯU TIÊN PHÉP TOÁN TRONG PYTHON (2)

8	<code><= < > >=</code> Phép toán so sánh
9	<code><> == !=</code> Phép toán so sánh bằng và khác nhau
10	<code>= %= /= //= -= += *= **=</code> Các phép toán chuyển nhượng (phép gán)
11	<code>is, is not</code> Phép toán nhận dạng
12	<code>in, not in</code> Phép toán thành viên
13	<code>not, or, and</code> Phép toán logic

2. BIẾN TRONG PYTHON

- ▶ Biến hiểu đơn giản là các vị trí bộ nhớ dành riêng để lưu trữ giá trị. Điều này có Nghĩa là khi bạn tạo 1 biến tức là bạn dành một khoảng trống trong bộ nhớ để lưu trữ.
- ▶ Dựa trên kiểu dữ liệu của một biến, trình thông dịch phân bổ bộ nhớ và quyết định những gì có thể lưu trữ trong bộ nhớ đó.

2.1. GÁN CÁC GIÁ TRỊ CHO CÁC BIẾN

- Các biến Python không cần khai báo rõ ràng để dự trữ dung lượng bộ nhớ. Dấu = được sử dụng để gán hay lưu giá trị cho các biến.

```
counter = 100      # Một số nguyên
miles   = 1000.0   # Một số thực
name    = "John"   # Một chuỗi

print(counter)
print(miles)
print(name)
```

2.2. GÁN LIÊN TIẾP

- ▶ Python cho phép bạn gán một giá trị duy nhất cho một số lượng biến đồng thời.
 - ▶ Ví dụ: `a = b = c = 1`
- ▶ Python cho phép bạn gán nhiều đối tượng cho nhiều biến
 - ▶ Ví dụ: `x, y, z = 1, 2, "John"`

2.3. KIỂU DỮ LIỆU CHUẨN

- ▶ Python có các kiểu dữ liệu chuẩn như sau:
 - ▶ Numbers
 - ▶ String
 - ▶ List
 - ▶ Tuple
 - ▶ Dictionary

2.3.1 KIỂU DỮ LIỆU NUMBERS (1)

- ▶ **Python có 4 loại số khác nhau:**

- ▶ int : Số nguyên có dấu
- ▶ long : Số nguyên dài, có thể biểu diễn dưới dạng bát phân và thập lục phân
- ▶ float : Giá trị thực dấu phẩy động
- ▶ complex : Số phức

- ▶ **Gán kiểu số lưu trữ giá trị số**

- ▶ `x = 5; y = 10`
- ▶ Thực hiện phép tính: `x+y`; `x*y`; `y/x`; `y-x`
- ▶ Gán tiếp `y=20`

- ▶ **Xóa biến**

- ▶ `del x`
- ▶ `del x, y`

2.3.1 KIỂU DỮ LIỆU NUMBERS (2)

int	long	float	complex
10	51924361L	0.0	3.14j
100	-0x19323L	15.20	45.j
-786	0122L	-21.9	9.322e-36j
080	0xDEFA BCECBDAECBFBAEI	32.3 + e18	.876j
-0490	535633629843L	-90.	-.6545 + 0J
-0x260	-052318172735L	-32.54e100	3e + 26J
0x69	-4721885298529L	70.2-E12	4.53e-7j

- Python cho phép bạn sử dụng chữ thường chữ l dài, nhưng bạn chỉ nên sử dụng chữ L viết hoa để tránh nhầm lẫn với số 1. Python hiển thị số nguyên dài có chữ hoa chữ L
- Một số phức bao gồm một cặp số có dấu phẩy động thực được ký hiệu là $x + yj$, trong đó x và y là số thực và j là đơn vị ảo.

2.3.2 KIỂU DỮ LIỆU STRING

```
str = 'Hello World!'

print(str)          # In ra chuỗi
print(str[0])       # In ra ký tự đầu của chuỗi
print(str[2:5])     # In ra các ký tự từ thứ 3 đến thứ 5 của chuỗi
print(str[2:])      # In ra chuỗi bắt đầu từ ký tự thứ 3
print(str * 2)      # In chuỗi 2 lần
print(str + "TEST") # Nối chuỗi xong rồi in
```

```
Hello World!
H
llo
llo World!
Hello World!Hello World!
Hello World!TEST
```

2.3.3 KIỂU DỮ LIỆU LIST

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]  
tinylist = [123, 'john']
```

```
print(list)           # In ra danh sách  
print(list[0])        # In ra phần tử đầu tiên của danh sách  
print(list[1:3])      # In ra các phần tử thứ 2 và 3 của danh sách  
print(list[2:])       # In ra các phần tử bắt đầu từ phần tử thứ 3 của danh sách  
print(tinylist * 2)   # In danh sách 2 lần  
print(list + tinylist) # Nối danh sách rồi in
```

```
['abcd', 786, 2.23, 'john', 70.2]  
abcd  
[786, 2.23]  
[2.23, 'john', 70.2]  
[123, 'john', 123, 'john']  
['abcd', 786, 2.23, 'john', 70.2, 123, 'john']
```

2.3.4 KIỂU DỮ LIỆU TUPLE

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2  )
tinytuple = (123, 'john')

print(tuple)           # In ra danh sách
print(tuple[0])        # In phần tử đầu tiên của danh sách
print(tuple[1:3])      # In các phần tử từ thứ 2 đến thứ 3 của danh sách
print(tuple[2:])       # In ra các phần tử bắt đầu từ phần tử thứ 3
print(tinytuple * 2)   # In danh sách 2 lần
print(tuple + tinytuple) # Nối danh sách rồi in
```

```
('abcd', 786, 2.23, 'john', 70.2)
abcd
(786, 2.23)
(2.23, 'john', 70.2)
(123, 'john', 123, 'john')
('abcd', 786, 2.23, 'john', 70.2, 123, 'john')
```

2.3.5 KIỂU DỮ LIỆU DICTIONARY

```
dict = {}  
dict['one'] = "This is one"  
dict[2]      = "This is two"  
  
tinydict = {'name': 'john', 'code':6734, 'dept': 'sales'}  
  
print(dict['one'])      # In giá trị của key 'one'  
print(dict[2])          # In giá trị của key 2  
print(tinydict)         # In toàn bộ dictionary  
print(tinydict.keys())  # In tất cả các key của từ điển  
print(tinydict.values()) # In tất cả các giá trị của từ điển
```

```
This is one  
This is two  
{'dept': 'sales', 'code': 6734, 'name': 'john'}  
['dept', 'code', 'name']  
['sales', 6734, 'john']
```

CÁM ƠN !

